IoT Intrusion Detection: Project Report

Authors: Serena Saba, Rodolph Neaimeh, Ibrahim Ibrahim, Emilio Sakr
Course: Machine Learning: Cybersecurity Project
Repository: https://github.com/Emilio-Sakr/iot-intrusion-detection

Data source

We use CIC-IoT-2023 (Canadian Institute for Cybersecurity, University of New Brunswick), a benchmark of network flows captured from 105 real IoT devices and labelled across 34 classes: BenignTraffic plus 33 attack types covering DDoS (12 sub-classes), DoS (4), Mirai (3), Recon (4), MITM/Spoofing, brute-force/scan, and six web/application-layer attacks. From the raw ~47 M-flow corpus we build a representative 5.8 M-row prepared sample using a two-pass streaming downsampler that caps each class at the median count while preserving every row of the rare classes. The resulting class imbalance is roughly 230 : 1 between the largest class (~286 K rows) and the smallest (Uploading_Attack, 1 252 rows), and this single fact shapes every modelling decision below.

Approach

All three models share an identical preprocessing pipeline and an identical stratified 70 / 15 / 15 train/val/test split (4 086 646 / 875 710 / 875 710 rows), so any difference between them reflects model capacity rather than data leakage or split mismatch. The preprocessor is fitted only on the training split and reused unchanged at inference, eliminating training/serving skew. The final deployed model is a three-stage hierarchical Random Forest that decomposes the 34-class problem into Benign-vs-Attack → Family → Specific class, served via a Dockerised FastAPI endpoint.

Methods

- **Preprocessing:** median imputation, Yeo-Johnson power transform on 19 skewed continuous features (selected by a three-gate rule: |skew| > 1, ≥ 50 unique values, ≥ 5 % non-zero), then standard scaling on all features; fitted on training rows only.
- **Imbalance handling:** balanced class weights w_c = N / (K · n_c), ranging from 0.60 on majority DDoS classes to 137.21 on Uploading_Attack; no oversampling, since SMOTE-style interpolation between binary protocol flags and integer packet counts produces flows that could not occur in real traffic.
- **Logistic Regression (baseline):** multinomial softmax with L-BFGS, C = 2.0, class_weight = "balanced", trained on all 4 086 646 rows.
- **Random Forest:** 200 trees, max_features = "sqrt", min_samples_leaf = 5, class_weight = "balanced_subsample", trained on the full training split for an apples-to-apples comparison with the three-stage model.
- **Three-stage RF (final):** Stage 1 binary RF (Benign vs Attack, threshold 0.30); Stage 2 family RF over 7 families (DDoS, DoS, Mirai, Recon, Spoofing, BruteScan, Web); Stage 3 per-family specialist RF; same RF hyperparameters at every stage; chained inference exposes sklearn-compatible predict / predict_proba.
- **Serving:** FastAPI service exposes /health, /predict, /predict/batch; the fitted preprocessor is applied server-side so clients send raw flow features; the Dockerfile bakes the trained joblib into the image; sub-50 ms per-request latency.

Results

Evaluated on the held-out test split (875 710 rows).
Primary metric is macro-F1.

Model	Training rows	Test accuracy	Test macro-F1
Logistic Regression	4 086 646	0.7800	0.5727
Random Forest	4 086 646	0.9670	0.8635
Three-stage RF (final)	**4 086 646**	**0.9687**	**0.8644**

Validation-to-test macro-F1 deltas are within ±0.002 on every model, confirming that the stratified split preserves class proportions across both held-out splits and that no hyperparameter choice was overfit to the validation set. The remaining error is concentrated in the Web family (XSS F1 = 0.45, Uploading_Attack F1 = 0.48, BrowserHijacking F1 = 0.52) a dataset-level ceiling, since at flow-statistics granularity these attacks are nearly indistinguishable from legitimate HTTP.

Conclusion

A disciplined, imbalance-aware preprocessing pipeline plus a hierarchical decomposition produces a deployable IDS at 96.87 % accuracy and 0.8644 macro-F1 on the held-out test set, served through a Dockerised FastAPI endpoint with sub-50 ms per-request latency. The three-stage architecture's main practical advantage at this data budget is deployment efficiency: matched macro-F1 in a model 60 % smaller (3.2 GB vs 8.0 GB).